

Resumen segundo parcial

Conceptos básicos

Validación vs verificación

La **validación** es una actividad **informal** que se realiza para comprobar que el software cumple con los requisitos del cliente, mientras que la **verificación** es una actividad **formal** que se realiza para comprobar que el software cumple con los requisitos especificados.

Especificación vs programa

La **especificación** es una **descripción** detallada de lo que el software debe hacer, mientras que el **programa** es la **implementación** concreta del software que se ejecuta en una computadora.

Formas de verificación

- **Semántica:** se usa la semántica de las instrucciones del lenguaje.
- **Sintáctica:** se usan axiomas y reglas de un método deductivo $\rightarrow$ Lógica de Hoare.

Lógica de Hoare

Definiciones

- **Terna de hoare:** $\{p\}C\{q\}$
 - p es la precondition.
 - C es el programa o fragmento de código a verificar.
 - q es la postcondición.
- **Precondición:** p
 - Es un predicado que se asume verdadero antes de ejecutar el programa.
- **Postcondición:** q
 - Es un predicado que se asume verdadero después de ejecutar el programa.
- **Especificación:** (p, q)
 - Es el par de precondition y postcondición.
- **Estado:** σ
 - Es una función que asigna a toda variable un valor. Por ejemplo, $\sigma(x) = 1$, $\sigma(y) = 2$, etc.

- **Satisfacción:**

- Un estado σ satisface a un predicado p , denotado $\sigma \models p$, si y solo si p es verdadero cuando se evalúa con los valores asignados por σ a sus variables.
- Por ejemplo, si p es $x < y$ y $\sigma(x) = 1, \sigma(y) = 2$, entonces $\sigma \models p$ porque $1 < 2$ es verdadero.

- **Correctitud:**

- Un programa S es correcto con respecto a una especificación (p, q) , denotado $\{p\}S\{q\}$, si y solo si para todo estado σ , si $\sigma \models p$ entonces S ejecutado a partir de σ se detiene en un estado σ' tal que $\sigma' \models q$.
- Si desde σ_0 se ejecuta S y no alcanza un estado σ'_0 dentro de q entonces se cumple $\{p\}S\{q\}$ porque si $\sigma \not\models q$, $\{p\}S\{q\}$ es trivialmente verdadero.

Lenguaje de programación (programas con while):

- Instrucciones:

- $S :: x := e$
- $S :: S_1; S_2$
- $S :: \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}$
- $S :: \text{while } B \text{ do } S_1 \text{ od}$

- Expresiones de tipo entero:

- $e :: n$
- $e :: x$
- $e :: (e_1 + e_2)$
- $e :: (e_1 - e_2)$
- $e :: (e_1 \cdot e_2)$
- Donde n es una constante entera y x es una variable entera.

- Expresiones de tipo booleano:

- $B :: \text{true}$
- $B :: \text{false}$
- $B :: (e_1 = e_2)$
- $B :: (e_1 < e_2)$
- ...
- $B :: \neg B_1$
- $B :: (B_1 \vee B_2)$
- $B :: (B_1 \wedge B_2)$
- ...

Lenguaje de especificación (lógica de predicados):

- $p :: \text{true}$
- $p :: \text{false}$
- $p :: (e_1 = e_2)$
- $p :: (e_1 < e_2)$
- ...
- $p :: \neg p$
- $p :: (p_1 \vee p_2)$

- $p :: (p_1 \wedge p_2)$
- ...
- $p :: \exists x : p$
- $p :: \forall x : p$

Axiomática y reglas

Axioma de la asignación (ASI)

- $\{p[x \mid e]\}x := e \{p\}$
- Se lee de derecha a izquierda: si luego de ejecutar $x := e$ vale p en términos de x , entonces antes de ejecutar $x := e$ valía p en términos de e .
- Por ejemplo:
 - $\{?\}x := x + 1 \{x \geq 0\}$
 - $\{x \geq 0[x \mid x + 1]\}x := x + 1 \{x \geq 0\}$
 - $\{x + 1 \geq 0\}x := x + 1 \{x \geq 0\}$
 - Es decir, si luego de $x := x + 1$ vale $x \geq 0$, entonces antes de $x := x + 1$ valía $x + 1 \geq 0$.

Regla de la secuencia (SEC)

- $\frac{\{p\}S_1\{r\}, \{r\}S_2\{q\}}{\{p\}S_1; S_2\{q\}}$
- El predicado r actúa como un nexo entre S_1 y S_2 y luego se descarta, no se propaga.
- Se puede generalizar a n instrucciones.
- Por ejemplo:
 - $\{p\}S_1; S_2; S_3\{q\}$ se puede descomponer en:
 - $\{p\}S_1\{r_1\}$
 - $\{r_1\}S_2\{r_2\}$
 - $\{r_2\}S_3\{q\}$

Regla del condicional (COND)

- $\frac{\{p \wedge B\}S_1\{q\}, \{p \wedge \neg B\}S_2\{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$
- Formula un modo de verificar una selección condicional fijando un único punto de entrada y un único punto de salida, correspondientes a p y q respectivamente.
- Por ejemplo:
 - $\{x \geq 0\} \text{ if } x > 0 \text{ then } y := 1 \text{ else } y := 0 \text{ fi } \{y \geq 0\}$
 - $\{x \geq 0 \wedge x > 0\}y := 1 \{y \geq 0\}$
 - $\{x \geq 0\}y := 1 \{y \geq 0\}$
 - $\{y \geq 0[y \mid 1]\}y := 1 \{y \geq 0\}$
 - $\{1 \geq 0\}y := 1 \{y \geq 0\}$
 - $\{true\}y := 1 \{y \geq 0\}$
 - $\{x \geq 0 \wedge \neg(x > 0)\}y := 0 \{y \geq 0\}$
 - $\{x \geq 0 \wedge x \leq 0\}y := 0 \{y \geq 0\}$
 - $\{x = 0\}y := 0 \{y \geq 0\}$
 - $\{y \geq 0[y \mid 0]\}y := 0 \{y \geq 0\}$

- $\{0 \geq 0\}y := 0\{y \geq 0\}$
- $\{true\}y := 0\{y \geq 0\}$

Regla de la repetición (REP)

- $$\frac{\{p \wedge B\}S\{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$
- p se define en términos de las variables del programa S .
- p es un predicado que vale antes y después de toda iteración, es decir, es un **invariante**.
- La regla NO asegura que el while termine.
- Si p vale al comienzo del bucle y mientras vale B el cuerpo S preserva p , entonces por razonamiento inductivo p vale al terminar el bucle.
- El procedimiento general para probar una terna con while es:
 - Encontrar un invariante.
 - Probar que el invariante vale antes de entrar al while: $Precondición \Rightarrow Invariante$.
 - Probar que se preserva el invariante en cada iteración: $\{Invariante \wedge CondiciónWhile\}CuerpoWhile\{Invariante\}$.
 - Probar que al terminar el while se cumple la postcondición: $Invariante \wedge \neg CondiciónWhile \Rightarrow Postcondición$.

Regla de la repetición con terminación (REP*)

- $$\frac{\langle p \wedge B \rangle S \langle p \rangle, \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$$
- Permite probar además la terminación del while. Se basa en un predicado invariante p y una función variante t que es entera y está definida, como el invariante, en términos de las variables del programa.
- Se le agregan dos premisas a REP y se reemplazan las llaves por corchetes angulares.
- En la segunda premisa, t se decrementa en cada iteración.
- Z es una variable lógica, no aparece en p , B , t ni en S .
- El objetivo de Z es fijar el valor de t antes de ejecutar S .
- En la tercera premisa, t se mantiene menor o igual a cero a lo largo de todo el bucle.
- Así, el bucle debe necesariamente terminar, porque en los números naturales no hay cadenas descendentes infinitas.
- El procedimiento general para probar una terna con while usando REP* es:
 - Probar su correctitud parcial con REP.
 - Encontrar una función variante t .
 - Probar que la función variante t se decrementa en cada iteración: $Invariante \wedge CondiciónWhile \Rightarrow t_{nuevo} < t_{viejo}$.
 - Probar que la función variante t está acotada inferiormente por cero: $t \geq 0$.

Regla de la consecuencia (CONS)

- $$\frac{p \rightarrow p_1, \{p_1\}S\{q_1\}, q_1 \rightarrow q}{\{p\}S\{q\}}$$
- Se puede reemplazar p_1 por un predicado p que lo implique.
- Se puede reemplazar q_1 por un predicado q al que implique.
- Permite reforzar precondiciones y debilitar postcondiciones:
 - De $\{x > 0\}S\{x = 0\}$ y $x > 5 \rightarrow x > 0$ se deduce $\{x > 5\}S\{x = 0\}$.
 - De $\{true\}S\{x = y + 1\}$ y $x = y + 1 \rightarrow x > y$ se deduce $\{true\}S\{x > y\}$.

- La regla no depende del lenguaje de programación si no del dominio semántico. Es una regla semántica más que sintáctica.
- Permite usar todos los axiomas del dominio semántico, como pueden ser los números enteros.

Separación de la prueba del while en dos

- Si bien la regla REP* permite probar tanto la correctitud parcial como la terminación de un while, es recomendable separar la prueba en dos partes:
 - Primero se prueba $\{p\}S\{q\}$ con REP para asegurar la invariancia de p .
 - Luego se prueba $\langle p \rangle S \langle true \rangle$ con REP* para asegurar la terminación del bucle.
 - La primera prueba se lee: "Si S termina a partir de p , lo hace en q ".
 - La segunda prueba se lee: " S termina a partir de p ".
 - Las dos pruebas puestas juntas se leen: " S termina a partir de p y lo hace en q ".

Composicionalidad

- El método de prueba presentado es **composicional**: dado un programa S , compuesto por subprogramas $S_1, S_2, \dots, S_n$, que valga la fórmula $\{p\}S\{q\}$, depende únicamente de que valgan las fórmulas $\{p_1\}S_1\{q_1\}$, $\{p_2\}S_2\{q_2\}, \dots, \{p_n\}S_n\{q_n\}$, sin importar el contenido de los subprogramas S_i (noción de **caja negra**).
- Por ejemplo, dado el programa $S :: S_1; S_2$, si se cumplen las fórmulas $\{p\}S_1\{r\}$ y $\{r\}S_2\{q\}$, entonces también se cumple la fórmula $\{p\}S_1; S_2\{q\}$, **independientemente del contenido de S_1 y S_2** .
- Si en vez de S_2 usamos un subprograma S_3 que también satisface $\{r\}S_3\{q\}$, entonces también se cumple la fórmula $\{p\}S_1; S_3\{q\}$, lo que implica que S_2 y S_3 son **intercambiables** (son funcionalmente equivalentes respecto de la tupla (r, q)).

Pérdida de la composicionalidad en los programas concurrentes

- En los programas concurrentes, la composicionalidad se pierde porque el orden de ejecución de los subprogramas no es fijo, sino que varía en cada ejecución. Por lo tanto, el resultado de ejecutar un subprograma puede depender del orden de ejecución de los subprogramas, lo que hace que no se puedan intercambiar subprogramas sin afectar el resultado final.
- Por ejemplo, si $\{x = 0\}S_1 :: x := x + 2\{x = 2\}$ y $\{x = 0\}S_2 :: z := x\{z = 0\}$, y se ejecutan de forma concurrente, entonces no siempre se cumple que $\{x = 0 \wedge x = 0\}[S_1 \parallel S_2]\{x = 2 \wedge z = 0\}$, porque si en el programa $S_1 \parallel S_2$ se ejecuta primero S_1 y luego S_2 , al final se cumple que $z = 2$ en vez de $z = 0$.
 - Lo que sí vale es $\{x = 0 \wedge x = 0\}[S_1 \parallel S_2]\{x = 2 \wedge (z = 0 \vee z = 2)\}$
 - Además, cambiar S_1 por un proceso equivalente $S_3 :: x := x + 1; x := x + 1$ no vale porque si S_2 se ejecuta entre medio de las dos asignaciones de S_3 , entonces al final se cumple que $z = 1$ en vez de $z = 0$ o $z = 2$.
Así, **en la concurrencia dos procesos funcionalmente equivalentes NO son intercambiables**.
 - Se pierde la noción de caja negra.

Especificaciones

- La especificación de un programa para calcular el factorial: $\{x > 0\}S_{fac}\{y = x!\}$ no es correcta, porque si bien el programa $S :: x := 1; y := 1$ satisface $(x > 0, y = x!)$, no es el programa pedido. Por ejemplo, si al empezar

$x = 5$, entonces al final debe ser $y = 5! = 120$ pero se tiene $y = 1$.

- El problema central es que las variables de la precondition pueden ser modificadas a lo largo del programa, efectivamente rompiendo la conexión entre la precondition y la postcondition. Para solucionar esto se introduce el concepto de **variable lógica**.
- Una variable lógica es una variable que aparece en la precondition pero no en el programa, es decir, no es modificada por el programa. Se usan para congelar valores.
- En el ejemplo dado, haríamos $(x = X \wedge X > 0, y = X!)$ con X la variable lógica.
- No es posible agregar a la especificación que la variable común x no se modifique nunca, ya que la lógica de predicados no lo permite. Una lógica que sí lo permite es la lógica temporal.

Sensatez, completitud y automatización de la verificación

- La **sensatez** es la propiedad obligatoria de que si una fórmula $\{p\}S\{q\}$ se cumple sintácticamente (con axiomas y reglas) entonces también se cumple semánticamente (considerando estados y computaciones).
- La **completitud** es la propiedad deseable de que si una fórmula $\{p\}S\{q\}$ se cumple semánticamente entonces se puede probar sintácticamente.
 - Su cumplimiento o no depende de la expresividad del lenguaje de especificación.
- La **automatización** no es posible porque la verificación de programas es indecidible en general y por lo tanto no automatizable. Sin embargo, existen muchas herramientas que asisten interactivamente al programador en la verificación de programas. En particular, si los programas son de estados finitos, existen sistemas de verificación automática denominados sistemas de model checking, que usan la lógica temporal y no son sintácticamente orientados como la Lógica de Hoare.
 - Herramientas de verificación axiomática: Dafny, COQ, Isabelle.
 - Herramientas de model checking: EMC, CAESAR, SMV, SPIN.
 - Herramientas de síntesis de programas: búsqueda estocástica, IA.

Correctitud parcial vs total

Parcial

- La **correctitud parcial** de un programa S con respecto a una especificación (p, q) , denotada $\{p\}S\{q\}$, significa que si S se ejecuta a partir de un estado σ que satisface p , entonces si S termina, lo hace en un estado σ' que satisface q . No se garantiza que S termine.
- Se puede visualizar vía una tabla de verdad donde:
 - p : el estado inicial satisface p .
 - r : S termina.
 - q : el estado final satisface q .

p	q	r	$(p \wedge r)$	$(p \wedge r) \rightarrow q$
V	V	V	V	V

p	q	r	$(p \wedge r)$	$(p \wedge r) \rightarrow q$
V	V	F	F	V
V	F	V	V	F
V	F	F	F	V
F	V	V	F	V
F	V	F	F	V
F	F	V	F	V
F	F	F	F	V

- Se puede ver que el único caso donde la correctitud parcial no se cumple es cuando el programa se ejecuta a partir de un estado que satisface p , el programa termina, pero lo hace en un estado que no satisface q .

Total

- La **correctitud total** de un programa S con respecto a una especificación (p, q) , denotada $\langle p \rangle S \langle q \rangle$, significa que si S se ejecuta a partir de un estado σ que satisface p , entonces S termina en un estado σ' que satisface q . Se garantiza que S termine.
- Se puede visualizar vía una tabla de verdad donde:
 - p : el estado inicial satisface p .
 - r : S termina.
 - q : el estado final satisface q .

p	q	r	$(r \wedge q)$	$p \rightarrow (r \wedge q)$
V	V	V	V	V
V	V	F	F	F
V	F	V	F	F
V	F	F	F	F
F	V	V	V	V
F	V	F	F	V
F	F	V	F	V
F	F	F	F	V

- Se puede ver que hay 2 casos donde la correctitud total no se cumple:
 - Cuando el programa se ejecuta a partir de un estado que satisface p , pero el programa no termina, por lo cual no importa si la postcondición se cumple o no.

- Cuando el programa se ejecuta a partir de un estado que satisface p , el programa termina, pero lo hace en un estado que no satisface q .

Sensatez y completitud de H y H^*

Sensatez de H

No puede pasar que **Sí se pueda demostrar** en H , por ejemplo, $\{x = 0\}x := x + 1\{x = 2\}$, porque ocurre que la precondition es verdadera, pero luego de ejecutar el programa no se cumple la postcondición.

Completitud de H

No puede pasar que **NO se pueda demostrar** en H , por ejemplo, $\{x = 0\}y := x\{y = 0\}$, porque ocurre que la precondition es verdadera y luego de ejecutar el programa se cumple la postcondición, por lo tanto la terna es válida.

Sensatez de H^*

No puede pasar que **Sí se pueda demostrar** en H^* , por ejemplo, $\langle x = 0 \rangle \text{ while } x = 0 \text{ do } skip \text{ od } \langle true \rangle$, porque ocurre que la precondition es verdadera, pero el programa no termina.

Completitud de H^*

No puede pasar que **NO se pueda demostrar** en H^* , por ejemplo, $\langle x = 0 \rangle \text{ while } x < 10 \text{ do } x := x + 1 \text{ od } \langle x = 10 \rangle$,

Simbología de la sensatez y completitud

- $\vdash_H \{p\}S\{q\}$ expresa que en el sistema H se puede demostrar la fórmula $\{p\}S\{q\}$.
- $\models \{p\}S\{q\}$ expresa que la fórmula $\{p\}S\{q\}$ se cumple **semánticamente**.
- $\vdash_{H^*} \langle p \rangle S \langle q \rangle$ expresa que en el sistema H^* se puede demostrar la fórmula $\langle p \rangle S \langle q \rangle$.
- $\models \langle p \rangle S \langle q \rangle$ expresa que la fórmula $\langle p \rangle S \langle q \rangle$ se cumple **semánticamente**.
- Se prueba que:
 - Si $\vdash_H \{p\}S\{q\}$ entonces $\models \{p\}S\{q\}$ (**sensatez** de H).
 - Si $\models \{p\}S\{q\}$ entonces $\vdash_H \{p\}S\{q\}$ (**completitud** de H).
 - Si $\vdash_{H^*} \langle p \rangle S \langle q \rangle$ entonces $\models \langle p \rangle S \langle q \rangle$ (**sensatez** de H^*).
 - Si $\models \langle p \rangle S \langle q \rangle$ entonces $\vdash_{H^*} \langle p \rangle S \langle q \rangle$ (**completitud** de H^*).
- En términos simples:
 - Las fórmulas que demuestran los sistemas H y H^* son verdaderas $\rightarrow$ **Sensatez**.
 - Todas las fórmulas que son verdaderas se pueden demostrar en los sistemas H y H^* $\rightarrow$ **Completitud**.

Inducción matemática

- Sea P una propiedad que se quiere probar en $\mathbb{N}$.
- Se compone de 3 pasos fundamentales: base inductiva, hipótesis inductiva y paso inductivo.
- **Base inductiva:** Si se cumple que $P(0)$ es verdadera (donde 0 es el elemento mínimo de $\mathbb{N}$)
- **Hipótesis inductiva:** Supongamos que $P(k)$ es verdadera para algún $k \in \mathbb{N}$.
- **Paso inductivo:** Y además que $\forall k \in \mathbb{N}, P(k) \rightarrow P(k+1)$, entonces se concluye que para todo $n = 0, 1, 2, \dots$, $P(n)$ es verdadera.
- Ejemplo de demostración de la sumatoria de n números naturales usando inducción matemática:
 - $\sum_{i=0}^n i = \frac{n(n+1)}{2}$
 - **Base inductiva:** $P(0) : \sum_{i=0}^0 i = \frac{0(0+1)}{2}$, es decir, $0 = 0$, lo cual es verdadero.
 - **Hipótesis inductiva:** Supongamos que $P(k) : \sum_{i=0}^k i = \frac{k(k+1)}{2}$ es verdadera para algún $k \in \mathbb{N}$.
 - **Paso inductivo:** Entonces, $\sum_{i=0}^{k+1} i = \sum_{i=0}^k i + (k+1)$. Por hipótesis inductiva, esto es igual a $\frac{k(k+1)}{2} + (k+1)$. Factorizando, se obtiene $\frac{k(k+1)+2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$, lo cual es igual a $\frac{(k+1)((k+1)+1)}{2}$, y por lo tanto se cumple que $\forall k \in \mathbb{N}, P(k) \rightarrow P(k+1)$.

Inducción estructural

- Es más general que la inducción matemática.
- Se usa para definir conjuntos y demostrar sus propiedades.
- Se aplica a cualquier dominio (los naturales, expresiones aritméticas, árboles, programas, etc).
- Puede haber varias bases inductivas y varios pasos inductivos.
- Ejemplo:
 - Definir por I.E las expresiones aritméticas con símbolos del conjunto $\{0, 1, x, +, \cdot, (,)\}$
 - **Bases inductivas:** 0, 1 y x son expresiones aritméticas.
 - **Pasos inductivos:** si e y f son expresiones aritméticas, entonces $(e + f)$ y $(e \cdot f)$ también lo son.
 - Probar que $C(e) = 1 + O(e)$, donde C es la cantidad de 0, 1 y x , y O es la cantidad de operadores $+$ y $\cdot$.
 - **Bases inductivas:**
 - $C(0) = 1 + O(0)$
 - $C(1) = 1 + O(1)$
 - $C(x) = 1 + O(x)$
 - **Paso inductivo para el operador $+$:**
 - Supongamos que $C(e) = 1 + O(e)$ y $C(f) = 1 + O(f)$.
 - Entonces, $C(e + f) = C(e) + C(f) = (1 + O(e)) + (1 + O(f))$ (hipótesis)
 - Como $O(e + f) = O(e) + 1 + O(f)$, entonces $C(e + f) = 1 + O(e + f)$.

Semántica del lenguaje de programación

Conceptos básicos

- Para probar las propiedades de sensatez y completitud de H y H^* , primero hay que definir formalmente la semántica del lenguaje de programación.
- Dados un programa S y un estado inicial σ , a dicha instancia o configuración inicial (S, σ) se le asocia una computación $\pi(S, \sigma)$ que es la secuencia de instancias de la ejecución de S a partir de σ .
 - Por ejemplo: $(x := 0; y := 1; z := 2, \sigma) \rightarrow (y := 1; z := 2, \sigma[x|0]) \rightarrow (z := 2, \sigma[x|0][y|1]) \rightarrow (E, \sigma[x|0][y|1][z|2])$.
- $val(\pi(S, \sigma))$ es el estado final de la computación $\pi(S, \sigma)$. Si $\pi(S, \sigma)$ es infinita, entonces $val(\pi(S, \sigma)) = \perp$ (no se detiene).
 - Ejemplo 1: $val(\pi(\text{while true do skip od}, \sigma)) = \perp$.
 - Ejemplo 2: $val(\pi(x := 10; \text{while } x > 0 \text{ do } x := x - 1 \text{ od}, \sigma)) = \sigma[x|0]$.

Definición por inducción estructural

- El lenguaje de programación se define por inducción estructural mediante una relación $\rightarrow$:
- $(skip, \sigma) \rightarrow (E, \sigma)$
 - El *skip* se consume en un paso (es atómico) y no modifica el estado inicial. E es la continuación sintáctica vacía.
- $(x := e, \sigma) \rightarrow (E, \sigma[x|\sigma(e)])$
 - La asignación $x := e$ es atómica y el estado final es como el inicial salvo que el valor de x es el de la expresión e según σ .
- Para toda instrucción T , si $(S, \sigma) \rightarrow (S', \sigma')$ entonces $(S; T, \sigma) \rightarrow (S'; T, \sigma')$
 - La secuencia $S; T$ se ejecuta de izquierda a derecha. Consumido S , si no diverge (si no loopea), se ejecuta T .
- $(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma) \rightarrow (S_1, \sigma)$ si se cumple $\sigma(B) = \text{verdadero}$
- $(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma) \rightarrow (S_2, \sigma)$ si se cumple $\sigma(B) = \text{falso}$
 - La evaluación de la expresión B es atómica y no modifica el estado inicial. Su evaluación establece si se ejecuta S_1 o S_2 .
- $(\text{while } B \text{ do } S \text{ od}, \sigma) \rightarrow (S; \text{while } B \text{ do } S \text{ od}, \sigma)$ si se cumple $\sigma(B) = \text{verdadero}$
- $(\text{while } B \text{ do } S \text{ od}, \sigma) \rightarrow (E, \sigma)$ si se cumple $\sigma(B) = \text{falso}$
 - La evaluación de la expresión B es atómica y no modifica el estado inicial. Su evaluación establece si se ejecuta el cuerpo S o si se termina el bucle.

Definición por extensión de la semántica de las instrucciones

- De la definición por comprensión o inductiva de las instrucciones se puede derivar su definición por extensión.
- Por ejemplo, en la secuencia, una computación $\pi(S_1; S_2, \sigma)$ tiene tres formas posibles:
 - Una computación **infinita** $(S_1; S_2, \sigma_0) \rightarrow (T_1; S_2, \sigma_1) \rightarrow (T_2; S_2, \sigma_2) \rightarrow \dots$ cuando S_1 diverge a partir de σ_0 .
 - Una computación **infinita** $(S_1; S_2, \sigma_0) \rightarrow \dots \rightarrow (S_2, \sigma_1) \rightarrow (T_1, \sigma_2) \rightarrow (T_2, \sigma_3) \rightarrow \dots$ cuando S_1 termina a partir de σ_0 pero S_2 diverge a partir de σ_1 .

- Una computación **finita** $(S_1; S_2, \sigma_0) \rightarrow \dots \rightarrow (S_2, \sigma_1) \rightarrow \dots \rightarrow (E, \sigma_2)$ cuando S_1 termina a partir de σ_0 y S_2 termina a partir de σ_1 .
- De forma similar se pueden obtener las formas de las computaciones de los otros tipos de instrucciones.

Prueba de la sensatez del método H

- Se cumple, para todo programa S y toda especificación (p, q) que $\vdash_H \{p\}S\{q\} \rightarrow \models \{p\}S\{q\}$.
- Se prueba por inducción matemática fuerte, considerando la longitud de la prueba (1 o más pasos):
 - Base inductiva: los axiomas son verdaderos (pruebas de tamaño 1).
 - Paso inductivo: las reglas son sensatas, es decir, preservan la verdad de las premisas (pruebas de tamaño ≥ 2)
- Ejemplo 1: Prueba de que el axioma *skip* es verdadero.
 - Dado $\vdash \{p\}skip\{p\}$, se quiere probar $\models \{p\}skip\{p\}$.
 - Sea $\vdash \{p\}skip\{p\}$.
 - Sea $\sigma \models p$.
 - La semántica del *skip* es $(skip, \sigma) \rightarrow (E, \sigma)$.
 - Así, luego de un *skip* queda $\sigma \models p$.
 - Por lo tanto, $\models \{p\}skip\{p\}$.
- Ejemplo 2: Prueba de que la regla SEC es sensata (preserva la verdad de las premisas)
 - Dado $\vdash \{p\}S_1; S_2\{q\}$ hay que probar $\models \{p\}S_1; S_2\{q\}$.
 - Sea $\vdash \{p\}S_1; S_2\{q\}$.
 - Se cumple $\vdash \{p\}S_1\{r\}$ y $\vdash \{r\}S_2\{q\}$ para algún r .
 - Las pruebas $\vdash \{p\}S_1\{r\}$ y $\vdash \{r\}S_2\{q\}$ son más cortas que la prueba $\vdash \{p\}S_1; S_2\{q\}$, por lo tanto se puede aplicar inducción.
 - Por hipótesis inductiva: $\models \{p\}S_1\{r\}$ y $\models \{r\}S_2\{q\}$.
 - Sea $\sigma_1 \models p$.
 - La semántica de la secuencia es $(S_1; S_2, \sigma_1) \rightarrow \dots \rightarrow (S_2, \sigma_2) \rightarrow \dots \rightarrow (E, \sigma_3)$.
 - Así, como $\models \{p\}S_1\{r\}$ y $\models \{r\}S_2\{q\}$, entonces luego de $S_1; S_2$ queda $\sigma_3 \models q$.
 - Por lo tanto $\models \{p\}S_1; S_2\{q\}$.

Prueba de la sensatez del método H^*

- Solo hay que probar que la regla REP* es sensata.
- $\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle \rightarrow \models \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$.
- Sea $\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$.
- Se cumplen dos cosas:
 - $\vdash \langle p \wedge B \rangle S \langle p \rangle$
 - $\vdash \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0$

- Las pruebas $\vdash \langle p \wedge B \rangle S \langle p \rangle$ y $\vdash \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0$ son más cortas que la prueba $\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od} \langle p \wedge \neg B \rangle$, por lo tanto se puede aplicar inducción.
- Por hipótesis inductiva: $\models \langle p \wedge B \rangle S \langle p \rangle$ y $\models \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0$.
- Sea $\sigma_0 \models p$. Hay que probar que el while termina en un estado $\sigma_k \models \langle p \wedge \neg B \rangle$.
- Supongamos que el while loopea a partir de σ_0 .
- Así, se tiene la computación infinita $(\text{while } B \text{ do } S \text{ od}, \sigma_0) \rightarrow (\text{while } B \text{ do } S \text{ od}, \sigma_1) \rightarrow (\text{while } B \text{ do } S \text{ od}, \sigma_2) \rightarrow \dots$ y por hipótesis inductiva, siempre vale $p, t \geq 0$ y t se decrementa en cada iteración.
- Pero esto no puede pasar, porque si no habría una cadena descendente infinita de números naturales.
- Por lo tanto el while termina y lo hace en algún estado $\sigma_k \models \langle p \wedge \neg B \rangle$